CS102L Data Structures and Algorithms
Review - Matrix Operations

Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Spring 2026

Copyright @ 2026 Habib University

Contents

1 Review Exercises **3**

1.1 Math Crush . . . 3

1.1.1 Problem Description . . . 3

1.1.2 Function Description . . . 4

1.1.3 Hint . . . 4

1.1.4 Constraints . . . 4

1.1.5 Sample . . . 4

1.1.6 Testing . . . 4

1.2 Image Blurring . . . 4

1.2.1 Function Description . . . 4

1.2.2 Testing . . . 5

1.3 Getting (A Matrix) Into Shape! . . . 5

1.3.1 Problem Description . . . 5

1.3.2 Constraints . . . 6

1.3.3 Function Description . . . 7

1.3.4 Testing . . . 7

1.4 Rotate Elements Around a Given Matrix Element . . . 7

1.4.1 Problem Description . . . 7

1.4.2 Function Description . . . 7

1.4.3 Sample Interactions . . . 8

1.4.4 Testing . . . 8

1.5 Keyboard . . . 9

1.5.1 Challenge . . . 9

1.5.2 Sample Interaction . . . 9

1.5.3 Sample Explaination . . . 9

1.5.4 Constraints . . . 10

1.5.5 Testing . . . 10

1.6 Spiral . . . 10

1.6.1 Problem Description . . . 10

1.6.2 Constraints . . . 11

1.6.3 Sample . . . 11

1.6.4 Testing . . . 11

1.7 Snakes and Ladders . . . 12

1.7.1 Problem Description . . . 12

1.7.2 Function Description . . . 12

1.7.3 Constraints 12
1.7.4 Hint 12
1.7.5 Sample Case 1 12
1.7.6 Sample Case 2 13
1.7.7 Testing 14

Review Exercises

1.1 Math Crush

1.1.1 Problem Description

Abdul Ahad loves mathematics, and plays all the mobile games based on numbers. The latest game called Math Crush displays an n × n grid of integers and the object of the game is to find all the matching pairs. A matching pair occurs when the same number occurs in the grid in adjacent positions, horizontally, vertically, or diagonally. Once done, the score is the sum of all the numbers in the matching pairs.

12	21	36	36	11
12	32	19	36	12
71	11	17	17	34
62	17	49	53	16
62	39	18	49	16

This is a grid with `n==5`. Pairs are indicated with lines. Some numbers are part of 2 pairs, like the 17 in the middle row and the 36 in the top row. There are 8 pairs in this grid: (12, 12), (36, 36), (36, 36), (17, 17), (17, 17), (62, 62), (49, 49), and (16, 16). The score for this grid is therefore: 12 + 12 + 36 + 36 + 36 + 36 + 17 + 17 + 17 + 17 + 62 + 62 + 49 + 49 + 16 + 16 = 490.

1.1.2 Function Description

Write a function, `score`, which takes 2 arguments: an integer n , and grid which represents an n × n grid as a nested list with each inner list representing a row. It returns the score of the grid.

1.1.3 Hint

- It may help for the function to first populate a list of all pairs found in the grid and then compute the sum of all the pairs.
- Write and test your function in stages. First write it for horizontal pairs only, then add in vertical pairs, and finally, diagonal pairs.

1.1.4 Constraints

- isinstance(n, int) and isinstance(grid, list)
- n > 0
- all(isinstance(sub, list) and len(sub) == n for sub in grid)

1.1.5 Sample

```
>> score(5, [12, 21, 36, 36, 11, 12, 32, 19, 36, 12, 71, 11,
    17, 17, 34, 62, 17, 49, 53, 16, 62, 39, 18, 49, 16])
562
```

1.1.6 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q1.py
```

1.2 Image Blurring

1.2.1 Function Description

An image can be blurred by replacing the value of each pixel by the average of the pixel itself and its neighboring pixels.
Write a function, `blur_image()`, that takes an image in the form of a nested list, A, and blurs it by averaging each pixel value with four of its neighboring pixels (Top, Bottom, Right, and Left).
For example:

Input:

$$A = \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 20 & 20 & 20 & 20 & 20 \\ 80 & 80 & 80 & 80 & 80 \\ 60 & 60 & 60 & 60 & 60 \\ 70 & 70 & 70 & 70 & 70 \end{bmatrix}$$

Output:

$$B = \begin{bmatrix} 13.33 & 12.50 & 12.50 & 12.50 & 13.33 \\ 32.50 & 30.00 & 30.00 & 30.00 & 32.50 \\ 60.00 & 64.00 & 64.00 & 64.00 & 60.00 \\ 67.50 & 66.00 & 66.00 & 66.00 & 67.50 \\ 66.67 & 67.50 & 67.50 & 67.50 & 66.67 \end{bmatrix}$$

Where each B_{ij} is defined as

$$B_{ij} = \frac{A_{ij} + A_{i,j+1} + A_{i,j-1} + A_{i+1,j} + A_{i-1,j}}{5}$$

For example,

$$B_{12} = \frac{20 + 20 + 20 + 80 + 10}{5} = 30.00$$

Note: Not all neighboring elements may be available at every pixel position. Suitable boundary conditions must be applied accordingly.

1.2.2 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

1.3 Getting (A Matrix) Into Shape!

1.3.1 Problem Description

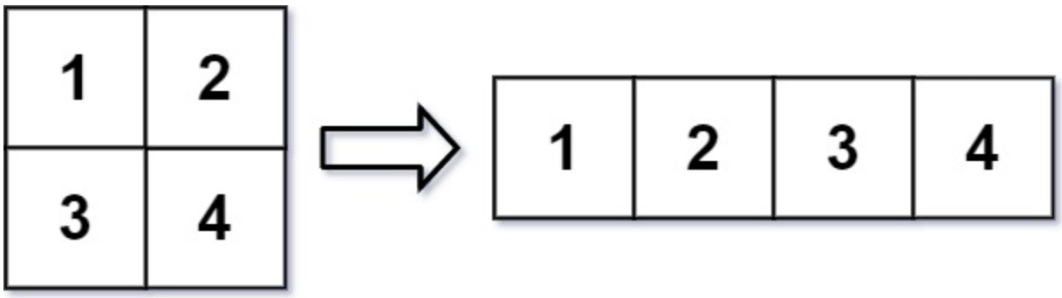
In MATLAB, there is a handy function called `reshape` which can reshape an $m \times n$ matrix into a new one with a different size $r \times c$ while keeping its original data.

You are given an $m \times n$ matrix `mat` and two integers r and c representing the number of rows and columns of the desired reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

If the `reshape` operation with the given parameters is possible and legal, output the new reshaped matrix; otherwise, output the original matrix.

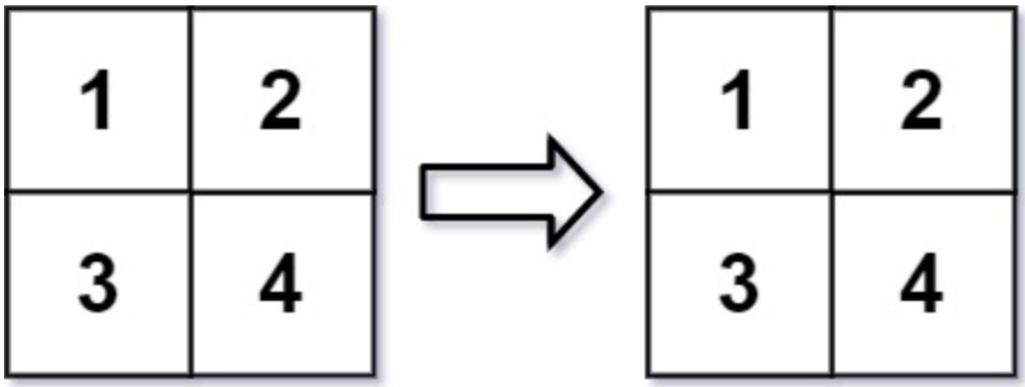
Example 1:




```
Input: mat = [[1,2],[3,4]], r = 1, c = 4
Output: [[1,2,3,4]]
```

Explanation Since $r = 1$ and $c = 4$, the original matrix should be reshaped to have one row and four columns. The output matrix does indeed have one row and four columns.

Example 2:




```
Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2],[3,4]]
```

Explanation Since the target number of rows is 2 and the target number of columns is 4, and there are not enough elements to fill that shape, the original matrix is returned.

1.3.2 Constraints

- $m = \texttt{len}(\texttt{mat})$
- $n = \texttt{len}(\texttt{mat}[0])$
- $1 \leq m, n \leq 100$

- $-1000 \leq \texttt{mat}[i][j] \leq 1000$
- $1 \leq r, c \leq 300$

1.3.3 Function Description

Write a function `matrixReshape` which takes three parameters: the matrix mat, the number of target rows r, and the number of target columns c.

1.3.4 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```

1.4 Rotate Elements Around a Given Matrix Element

1.4.1 Problem Description

You are given an $n \times n$ matrix of integers and a target integer. Your task is to rotate all elements surrounding the target value in an anti-clockwise direction by one position.

- The rotation includes up to 8 neighboring elements (top, bottom, left, right, and the 4 diagonals).
- The target element itself stays in place.
- Rotation should wrap around, meaning the last neighbor moves into the first neighbor's position.
- If the target element is on an edge or corner (so not all 8 neighbors exist), rotate only the existing neighbors.

1.4.2 Function Description

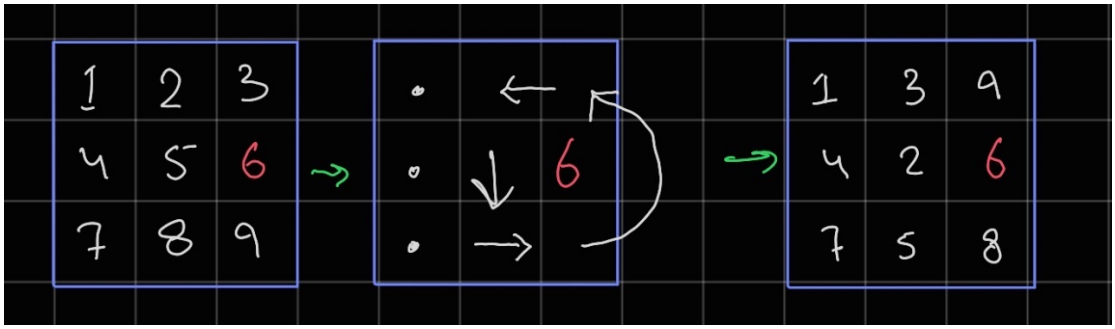
Write a function `rotation` that takes as a parameter matrix and target and rotates all elements surrounding the target value in the matrix in an anti-clockwise direction by one position. The function will not return anything and changes the matrix in-place.

1.4.3 Sample Interactions

Example 1

```
>> matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>> rotation(matrix, 5)
>> matrix
[[2, 3, 6], [1, 5, 9], [4, 7, 8]]
```

Example 2




```
>> matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>> rotation(matrix, 6)
>> matrix
[[1, 3, 9], [4, 2, 6], [7, 5, 8]]
```

1.4.4 Testing

In order to test your function, type the following command on the terminal:

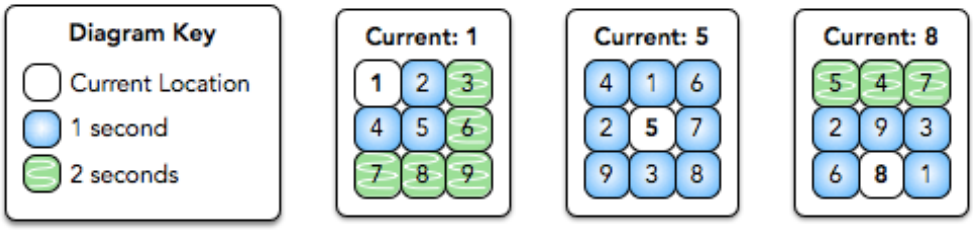
```
pytest tests/test_q4.py
```

1.5 Keyboard

1.5.1 Challenge

You work at a secret organization where you must type a string of numbers into a console using a 3×3 numeric keypad. Every day they mix up the numbers on the keypad.
Use the following rules to calculate the total amount of time it takes to type a string:

- It takes 0 seconds to move your finger to the first key, and it takes 0 seconds to press the key where your finger is located any number of times.
- You can move your finger from one location to any adjacent key in one second.
- Moving to a non-adjacent key is done as a series of moves to adjacent keys.



This diagram depicts the minimum amount of time it takes to move from the current location to all other locations on the keypad.

Write a function `entryTime` that takes two parameters s and keypad and returns an integer denoting the minimum amount of time it takes to type the string s. keypad is a string of 9 digits where each group of 3 digits represents a row on the keypad of the day, in order.

1.5.2 Sample Interaction

```
>>> entryTime('423692','923857614')
8
```

1.5.3 Sample Explaination

For the given sample, the keypad looks like this:

9	2	3
8	5	7
6	1	4

We calculate the time it takes to type s=423692 as follows:

- 4: We start at this number so it takes 0 seconds.
- 2: It takes 2 seconds to move from 4 => 2
- 3: It takes 1 second to move from 2 => 3
- 6: It takes 2 seconds to move from 3 => 6
- 9: It takes 2 seconds to move from 6 => 9
- 2: It takes 1 second to move from 9 => 2

The total time is $2 + 1 + 2 + 2 + 1 = 8$.

1.5.4 Constraints

- `isinstance(s, str) is True`
- keypad is a string of 9 digits where each group of 3 digits represents a row on the keypad of the day, in order

1.5.5 Testing

In order to test your function, type the following command on the terminal:

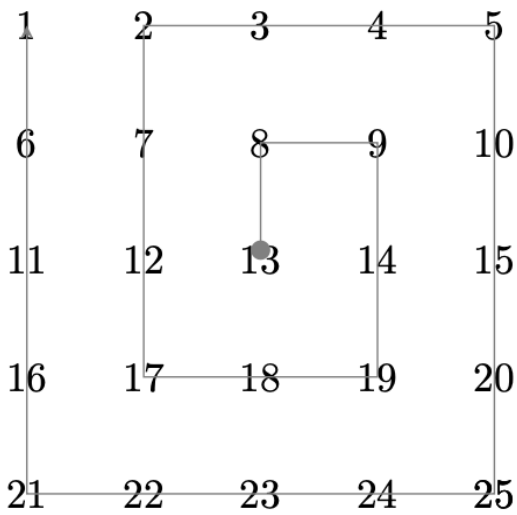
```
pytest tests/test_q5.py
```

1.6 Spiral

1.6.1 Problem Description

We are given a square matrix, `M`, of size `n` as a nested `list`. Each inner `list` represents a row of `M`. We want to traverse all the elements of `M` in spiral order starting form its center element, as pictured below.
This is a matrix with `n==5`. The traversal starts at the center element of the matrix and spirals outward. The traversal is the list of elements visited in this order. Namely,
`[13, 8, 9, 14, 19, 18, 17, 12, 7, 2, 3, 4, 5, 10, 15, 20, 25, 24, 23, 22, 21, 16, 11, 6, 1]`.

Write a function, `spiral`, that takes a square matrix, M, as described above. It returns a *list* containing all the elements of M in spiral order.



1.6.2 Constraints

- `isinstance(M, list)`
- `len(M) > 0 and len(M) % 2 == 1`
- `all(isinstance(row, list) and len(row) == len(M) for row in M)`

1.6.3 Sample

```
>>> matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> print(spiral(matrix))
[5, 2, 3, 6, 9, 8, 7, 4, 1]

>>> matrix = []
>>> for i in range(1, 22, 5):
        matrix.append(list(range(i, i+5))
>>> print(spiral(matrix))
[13, 8, 9, 14, 19, 18, 17, 12, 7, 2, 3, 4, 5, 10, 15, 20,
   25, 24, 23, 22, 21, 16, 11, 6, 1)]

>>> matrix = [[9]]
>>> print(spiral(matrix))
[9]
```

1.6.4 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q6.py
```

1.7 Snakes and Ladders

1.7.1 Problem Description

You are given an $n \times n$ matrix *board* representing a **Snakes and Ladders** game. The cells are labeled from 1 to n^2, starting from the bottom-left cell (`board[n-1][0]`), and the direction alternates in each row. An integer list *diceRolls* contains the result of a standard six-sided die rolled at each turn of the player.

You start on square 1 of the board. At each turn i, player move their counter forward to the number of spaces as shown on the *diceRolls[i]*. A board square on row r and column c has a snake or ladder if `board[r][c]` $\neq$ `-1`. The destination of that snake or ladder is the coordinate mentioned in `board[r][c]`. Squares 1 and n^2 do not have a snake or ladder.

1.7.2 Function Description

Implement a function `playSandL(board, diceRolls)` that returns a list of coordinates representing the player's position after each turn.

1.7.3 Constraints

- You must **not** use `append`, `insert`, `remove`, `pop`, or any similar operation that changes the size of the matrix.

1.7.4 Hint

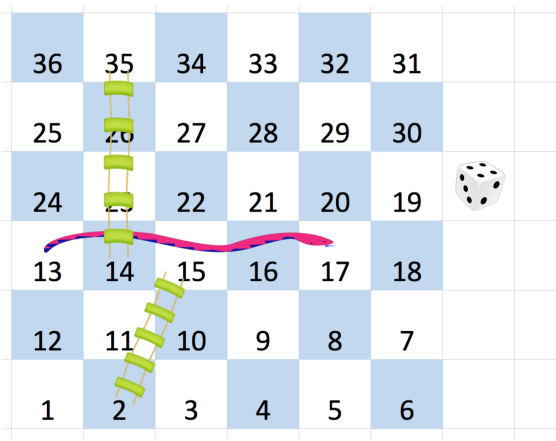
First implement the logic for even values of n. A small modification is required to support odd values of n.

1.7.5 Sample Case 1

```
>> board = [[-1, -1, -1, -1, -1, -1],
            [-1, -1, -1, -1, -1, -1],
            [-1, -1, -1, -1, -1, -1],
            [-1, (0, 1), -1, -1, (3, 0), -1],
            [-1, -1, -1, -1, -1, -1],
            [-1, (3, 2), -1, -1, -1, -1]]

>> diceRolls = [1, 2, 1, 1]

>> playSandL(board, diceRolls)
[(3, 2), (3, 0), (0, 1), (0, 0)]
```



Sample Case 1 Explanation

Starting Coordinate	Dice Roll	Destination	Explanation
(5, 0)	1	(3, 2)	Ladder at (5, 1) moves player to (3, 2)
(3, 2)	2	(3, 0)	Snake at (3, 4) moves player to (3, 0)
(3, 0)	1	(0, 1)	Ladder at (3, 1) moves player to (0, 1)
(0, 1)	1	(0, 0)	Normal move

1.7.6 Sample Case 2

```
>> board = [[-1, (3, 4), -1, (5, 5), -1, -1],
            [-1, (2, 1), -1, (0, 5), -1, -1],
            [-1, (0, 1), -1, (1, 3), -1, -1],
            [-1, (1, 5), (2, 0), -1, (3, 0), -1],
            [-1, (4, 2), -1, (3, 3), (1, 1), -1],
            [-1, -1, (1, 2), (3, 2), -1, -1]]

>> diceRolls = [1, 6, 6, 2, 1, 2]

>> playSandL(board, diceRolls)
[(5, 1), (3, 0), (2, 5), (0, 5), (0, 4), (0, 2)]
```

Sample Case 2 Explanation

$$\begin{bmatrix} -1 & (3,4) & -1 & (5,5) & -1 & -1 \\ -1 & (2,1) & -1 & (0,5) & -1 & -1 \\ -1 & (0,1) & -1 & (1,3) & -1 & -1 \\ -1 & (1,5) & (2,0) & -1 & (3,0) & -1 \\ -1 & (4,2) & -1 & (3,3) & (1,1) & -1 \\ -1 & -1 & (1,2) & (3,2) & -1 & -1 \end{bmatrix}$$

Starting Coordinate	diceRolls[i]	Destination Coordinate
(5, 0)	1	(5, 1)
(5, 1)	6	(4, 4) → (1, 1) → (2, 1) → (0, 1) → (3, 4) → (3, 0)
(3, 0)	6	(2, 5)
(2, 5)	2	(2, 2) → (1, 3) → (0, 5)
(0, 5)	1	(0, 4)
(0, 4)	2	(0, 2)

1.7.7 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q7.py
```